LearnifyEnglish

Architecture, idées et concepts
des deux modèles d'IA générative ancrés dans les cours (LLM + RAG)

	Modèle	En une phrase
1	**Ask the course**	Un assistant qui répond aux questions de l'étudiant **uniquement à partir du cours** et cite la leçon et le moment de la vidéo (ex. *[Leçon 3 – 04:12]*).
2	**Générateur de quiz ancré**	Des questions générées à partir des passages d'une leçon ; chaque réponse est prouvée par une citation exacte du passage, montré à l'étudiant s'il se trompe.

course-service (Spring Boot 3.3, Java 17) · Google Gemini · Angular 19 · MySQL

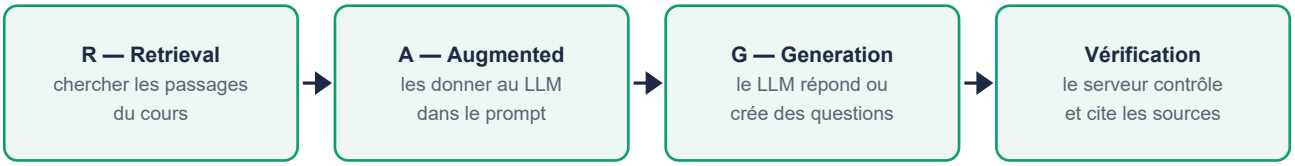
1. L'idée de départ

Un LLM (*Large Language Model*) comme Gemini sait rédiger, expliquer et poser des questions. Mais utilisé seul dans une plateforme de cours, il pose trois problèmes :

- **Il ne connaît pas le cours** : il répond avec ses connaissances générales, qui peuvent contredire ce que le professeur a dit dans la vidéo.
- **Il peut inventer** (hallucination) : une réponse fausse mais bien rédigée est dangereuse pour un étudiant.
- **Il ne dit pas d'où vient l'information** : l'étudiant ne peut ni vérifier, ni revoir le passage concerné.

L'idée commune aux deux modèles est le **RAG** (*Retrieval-Augmented Generation*, génération augmentée par la recherche) : avant d'appeler le LLM, on **cherche** dans le contenu du cours les passages utiles, puis on demande au LLM de **travailler uniquement sur ces passages**, et on **vérifie** sa sortie côté serveur.

Le principe RAG appliqué aux deux modèles



Principe directeur : ne pas faire confiance au modèle. Tout ce qui peut être calculé de façon fiable l'est par le serveur : le choix des passages, les timestamps, les numéros de page, les liens vers la vidéo, la structure des questions et la preuve que la réponse vient du passage. Le LLM ne fait que ce qu'il fait bien : comprendre et rédiger.

2. Les concepts utilisés

Concept	Explication	Où il sert
LLM	Modèle de langage qui génère du texte à partir d'un prompt. Ici : Gemini 2.5 Flash.	Réponses, quiz, transcription
Prompt système	Instructions fixes données au modèle (règles à respecter), séparées des données.	Les deux
Sortie structurée	Le modèle doit renvoyer un JSON conforme à un schéma (*responseSchema*) : le serveur peut le lire et le valider champ par champ.	Les deux
Transcription	Gemini « regarde » la vidéo YouTube et en écrit le texte, avec l'heure de début de chaque phrase (format MM:SS).	Indexation
Découpage (chunking)	Le contenu est coupé en passages d'environ 1000 caractères ; un passage garde ses métadonnées (cours, leçon, début dans la vidéo, page PDF).	Indexation
Embedding	Vecteur de 768 nombres qui représente le **sens** d'un texte : deux textes de sens proche ont des vecteurs proches, même avec des mots différents.	Recherche
Similarité cosinus	Mesure entre −1 et 1 de la proximité de deux vecteurs (angle entre eux).	Recherche
Top-k	On garde les k passages les plus proches (k = 5 pour une question).	Ask the course
Seuil de pertinence	Similarité minimale (0,55) : en dessous, le passage est jugé hors sujet. Si rien ne l'atteint, le LLM n'est pas appelé.	Les deux
MMR	*Maximal Marginal Relevance* : choisit des passages à la fois représentatifs et différents les uns des autres, pour éviter les doublons.	Quiz
Ancrage (grounding)	Obliger la sortie à reposer sur des passages précis, identifiés par [S1], [S2]…	Les deux
Citations côté serveur	Le serveur traduit [S2] en « Leçon 3 – 04:12 » grâce aux métadonnées : le modèle ne peut pas inventer un timestamp.	Les deux
Contrôle de fidélité	Le modèle doit citer mot pour mot la phrase qui prouve la réponse ; le serveur vérifie qu'elle est vraiment dans le passage.	Quiz
Injection de prompt	Attaque où un texte contient « ignore tes instructions ». Les extraits sont balisés, neutralisés et déclarés comme données.	Les deux
Idempotence	Réindexer un contenu inchangé ne coûte rien : chaque source a une empreinte SHA-256.	Indexation
CECRL	Niveaux européens A1 à C2 : le quiz adapte son vocabulaire au niveau choisi.	Quiz

3. Architecture globale

Le projet est organisé en microservices Spring Boot enregistrés dans Eureka et exposés par une API Gateway. Toute la partie IA est placée dans **course-service**, car c'est lui qui possède le contenu des cours. Les quiz générés sont enregistrés dans **quiz-feedback-service**, et l'identité de l'utilisateur vient du JWT émis par **user-service**.

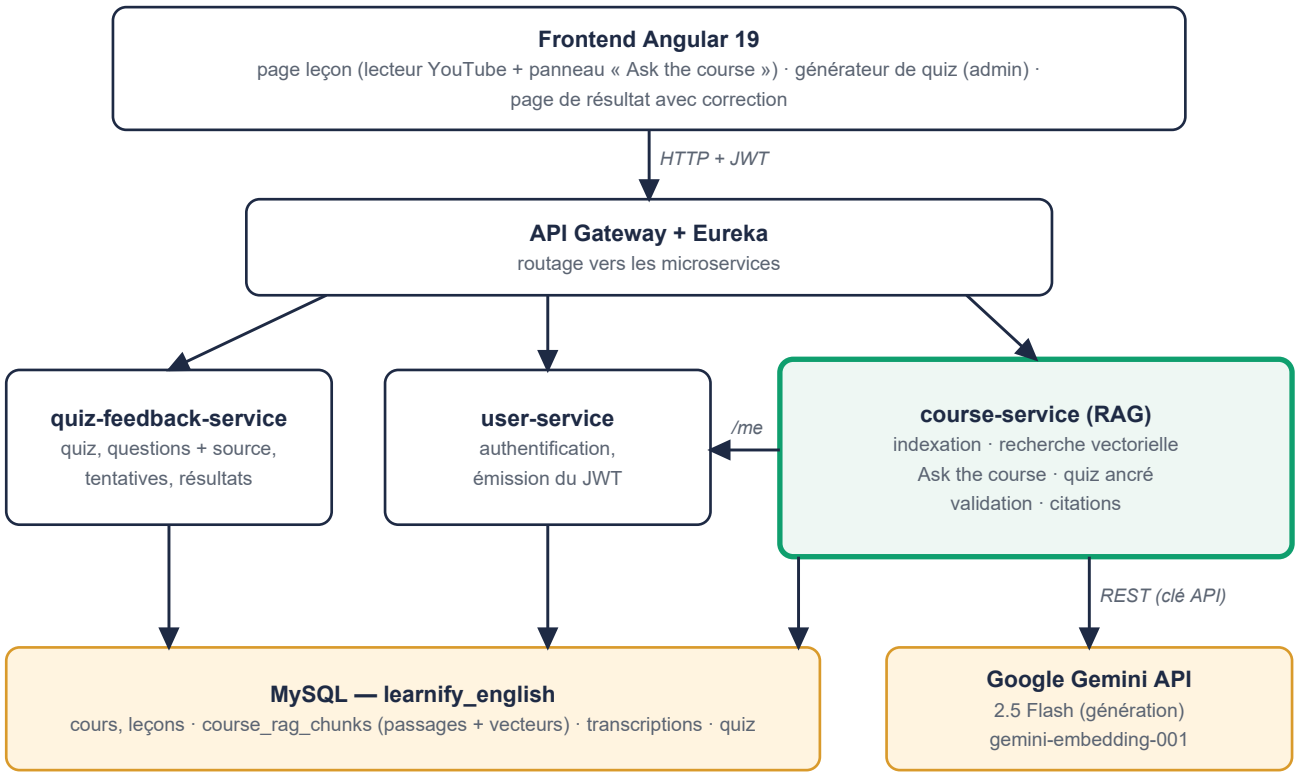


Figure 1 — Vue d'ensemble : le seul service qui appelle Gemini est course-service.

3.1 Architecture interne de course-service

Couche	Classes principales	Rôle
Contrôleurs REST	CourseAskController, CourseRagAdminController	Routes /ask (étudiant) et /admin/rag/* (administrateur)
Sécurité	JwtAuthFilter, SecurityConfig, AskRateLimiter	Lecture du JWT, rôle ADMIN, limite 5/min et 50/jour
Indexation	CourseRagIngestionService, LessonTranscriptionService, TranscriptChunker, PdfTextExtractor	Transcrire, découper, calculer les embeddings, stocker
Index vectoriel	CourseVectorIndex, CourseChunkStore	Passages + vecteurs en MySQL, cache mémoire par cours
Modèle 1	CourseRetrievalService, CourseAskService, CitationResolver	Recherche, seuil, génération, citations
Modèle 2	LessonPassageSelector, LessonQuizService, EvidenceMatcher	Sélection thème/MMR, génération, validation, fidélité
Client IA	GeminiClient	Appels REST à Gemini, reprises sur erreur 429
Événements	AutoReindexListener, StartupReindexRunner	Réindexation automatique après modification et au démarrage

4. Le socle commun : l'index du cours

Les deux modèles lisent le même index. Il est construit en tâche de fond, automatiquement, dès qu'une leçon est ajoutée ou modifiée.

Pipeline d'indexation

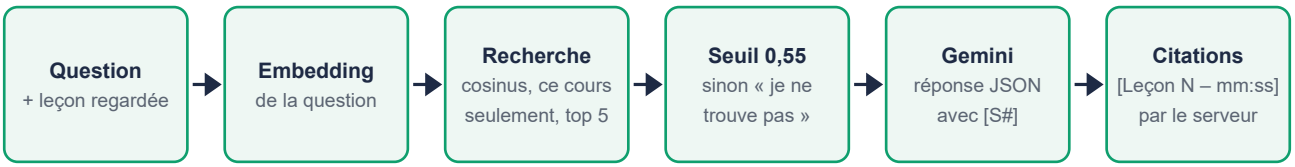


- **Métadonnées d'un passage** : courseId, lessonId, titre et ordre de la leçon, type de source (description, transcription, PDF), début et fin dans la vidéo, page du PDF.
- **Fiabilité des timestamps** : ils doivent être ordonnés et compris dans la durée de la vidéo ; une transcription incohérente est rejetée. Les vidéos longues sont transcrites par tranches de 10 minutes.
- **Un segment n'est jamais coupé** : le début d'un passage correspond toujours au début d'une phrase prononcée, ce qui donne des citations précises.

5. Modèle 1 — « Ask the course »

Idée : un assistant de cours qui répond comme le professeur, avec ses mots, et qui montre où il l'a dit. Un clic sur la citation relance la vidéo au bon moment.

Traitement d'une question



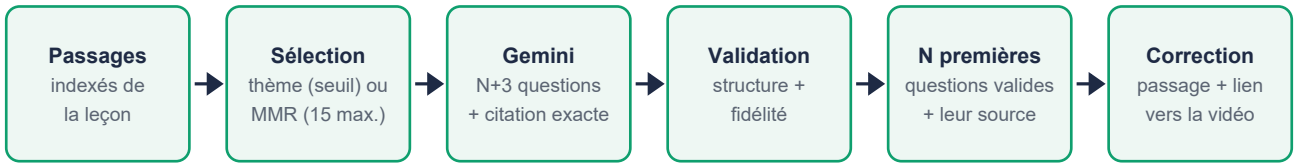
Étape	Concept appliqué	Détail
Recherche	Embedding + similarité cosinus + top-k	Seuls les passages du cours sont comparés ; ceux de la leçon regardée reçoivent un bonus de +0,05.
Filtre	Seuil de pertinence	Sous 0,55, réponse « Je ne trouve pas cette information dans le cours. » sans appel au LLM (économie et zéro hallucination).
Génération	Ancrage + sortie structurée	Le prompt impose : extraits seulement, une étiquette [S#] par affirmation, langue de la question, ignorer les instructions des extraits.
Post-traitement	Citations côté serveur	[S2] devient « [Leçon 1 – 02:21] » ; une réponse qui ne cite aucune source valide est refusée.

> **Exemple réel.** « What time does the alarm go off? » → passage le plus proche à **02:21** : *« …my alarm goes off at 6:30 a.m. and I jump out of bed… »*. Réponse citée **[Leçon 1 – 02:21]** ; le clic lance la vidéo à 2 min 21 s.

6. Modèle 2 — Générateur de quiz ancré dans les leçons

Idée : au lieu de questions « libres » sur un sujet, générer les questions **à partir de ce que dit réellement une leçon**. Chaque question garde son passage source : si l'étudiant se trompe, il voit la phrase exacte du cours et peut revoir le moment de la vidéo.

De la leçon à la correction



6.1 Récupération : choisir les bons passages

- **Avec un thème** (ex. « le petit-déjeuner ») : le thème est transformé en embedding et comparé aux passages de la leçon. On garde ceux qui dépassent le seuil de 0,55 (au plus 15). S'il y en a moins de 2, le serveur répond **422 « Ce thème n'est pas traité dans cette leçon. »** sans appeler le LLM.
- **Sans thème, leçon longue** (plus de 15 passages) : sélection **MMR**. La « requête » est le centroïde (moyenne normalisée) des vecteurs de la leçon. À chaque étape, on choisit le passage qui maximise `λ·sim(centroïde, p) − (1−λ)·max sim(p, déjà choisis)`, avec λ = 0,5 : représentatif de la leçon **et** différent des passages déjà pris. Le résultat est déterministe.
- **Leçon courte** : tous les passages sont utilisés.
- Dans tous les cas, les passages sont remis **dans l'ordre de la leçon** (description, puis vidéo) avant d'être étiquetés [S1]…[S15]. λ et le nombre de passages sont configurables.

6.2 Génération et validation stricte

Le modèle reçoit les passages, le niveau CECRL, le type de questions et éventuellement le thème. Le générateur utilise sa propre clé API Gemini (quota séparé de celui des étudiants). Côté serveur, une question est **rejetée** si :

Règle	Contrôle
Source	L'étiquette [S#] doit désigner un passage fourni ; pas de question en double.
QCM	Exactement 4 options, non vides et distinctes (sans tenir compte des espaces et de la casse) ; la bonne réponse correspond à exactement une option.
Vrai/Faux	Exactement 2 options True/False (Vrai/Faux accepté) ; la bonne réponse est l'une des deux.
Mixte	Chaque question est contrôlée selon son propre type.
Fidélité	Le champ *evidence* (citation de 5 à 40 mots) doit se trouver dans le passage cité.

6.3 Contrôle de fidélité : la réponse est-elle vraiment dans le passage ?

Une étiquette [S#] valide ne prouve pas que la réponse vient du passage. Le modèle doit donc recopier **mot pour mot** la phrase qui justifie la bonne réponse, et le serveur la vérifie de façon déterministe, sans second appel au LLM :

- **Normalisation** des deux textes : Unicode NFKC, minuscules, apostrophes et guillemets typographiques unifiés, ponctuation supprimée, espaces compactés.
- **Tolérance** : la citation est acceptée si au moins **80 %** de ses mots apparaissent, dans l'ordre, dans une même portion continue du passage (petites différences de recopie tolérées, citation inventée rejetée).

Sur-génération : comme certaines questions peuvent être rejetées, le modèle en génère **3 de plus** que demandé dans le même appel, et le serveur garde les N premières valides. La réponse de l'API indique le nombre de questions demandées, générées, retenues et rejetées (avec le motif), ainsi que le mode de sélection ; l'interface affiche « X questions retenues, Y rejetées ».

6.4 Correction pour l'étudiant

Chaque question enregistrée garde trois informations : le libellé de la source (ex. « Leçon 2 – 01:31 »), le lien vers la vidéo au bon moment et le texte du passage. Sur la page de résultat, chaque erreur affiche la bonne réponse, l'explication, le passage exact et un bouton « Voir ce passage de la vidéo ».

7. Les deux modèles côte à côte

	Ask the course	Quiz ancré
Utilisateur	Étudiant (connecté)	Administrateur (rôle ADMIN)
Entrée	Une question libre	Leçon, niveau CECRL, nombre et type de questions, thème optionnel
Périmètre de recherche	Tout le cours (bonus pour la leçon regardée)	Une seule leçon
Récupération	Top 5 par similarité + seuil 0,55	Thème (seuil 0,55) ou MMR (λ = 0,5, 15 passages)
Refus sans LLM	Aucun passage au-dessus du seuil	Moins de 2 passages sur le thème (HTTP 422)
Sortie du LLM	{answer, insufficient} avec [S#]	Questions avec source [S#] et citation exacte
Contrôle serveur	Citations valides obligatoires	Structure stricte + fidélité de la citation
Résultat	Réponse + citations cliquables vers la vidéo ou le PDF	Quiz enregistré ; correction avec passage et lien vidéo
Clé API	Clé principale	Clé dédiée au générateur

8. Tests et évaluation

- **95 tests automatisés réussis** (Gemini simulé, aucun appel réel) : découpage et timestamps, citations, classement et seuil, sélection par thème et MMR (déterminisme, ordre de la vidéo), validation stricte des questions, contrôle de la citation, sur-génération, sécurité, et un test d'intégration complet (Spring, base H2, JWT).
- **Évaluation avec le vrai Gemini** sur un cours de démonstration (4 leçons + 1 PDF), 10 questions dont 3 hors cours : **7/7 citations correctes et 3/3 refus** ; un extrait piégé « IGNORE ALL PREVIOUS INSTRUCTIONS » a été ignoré.

9. Limites et perspectives

- **Quota gratuit Gemini** (20 générations par jour et par modèle) : suffisant pour une démonstration, pas pour la production.
- **Timestamps écrits par le LLM** : validés, mais parfois décalés de quelques secondes ; un modèle de reconnaissance vocale dédié (Whisper) serait plus précis.
- **Seuil et λ** calibrés sur peu d'exemples : un jeu d'évaluation plus large permettrait de les ajuster.
- **Perspectives** : base vectorielle dédiée (pgvector, Qdrant), recherche hybride mots-clés + vecteurs, reranker, réponses en streaming, mesure de la qualité avec RAGAS.